

Lab 1 - EECS 211

Winter 2026

1 Overview

In this assignment, you will learn how to access the EECS server and use QEMU to emulate a RISC-V system and run a simple hello world program. This lab covers the following topics:

- Configuring Visual Studio Code with Remote-SSH to access EECS remote server
- Running assembly code on QEMU
- Using Makefiles to automate the building and execution process

Lab environment: All tasks in this lab are performed on EECS-provided Linux servers. Students will connect to the server and execute all commands in a terminal environment remotely.

Readings and references: Students are encouraged to consult the following resources for additional background:

- QEMU System Emulator Documentation
- RISC-V Instruction Set Manual
- GNU Make Documentation

Rubrics (100 points): Students are required to follow the steps and write a PDF report. The report should contain screenshots of terminal outputs and explanation of observations or interesting findings. Points will be deducted if no screenshots or sufficient explanations are given. The students should also **fill the honesty statement and upload it to canvas**, otherwise 30 points will be deducted.

Due date: 01/14/2026, 23:59:59PM PST

2 Background

In order to get things started, it is better to get familiar with what QEMU is. You can refer to the QEMU System Emulator Documentation to get more familiar with the QEMU emulator.

3 Setup

First, we need to access one of the EECS servers to be able to access the QEMU emulator. Complete the following steps:

1. Install an OpenSSH compatible SSH client on your system if one is not already present.
2. Install Visual Studio Code. You will be using this when working on the assignments on the server via SSH tunneling. <https://code.visualstudio.com/>
3. Install the **Remote-SSH** extension in VS Code. A screenshot of the extension installation page can be seen in Figure 1.

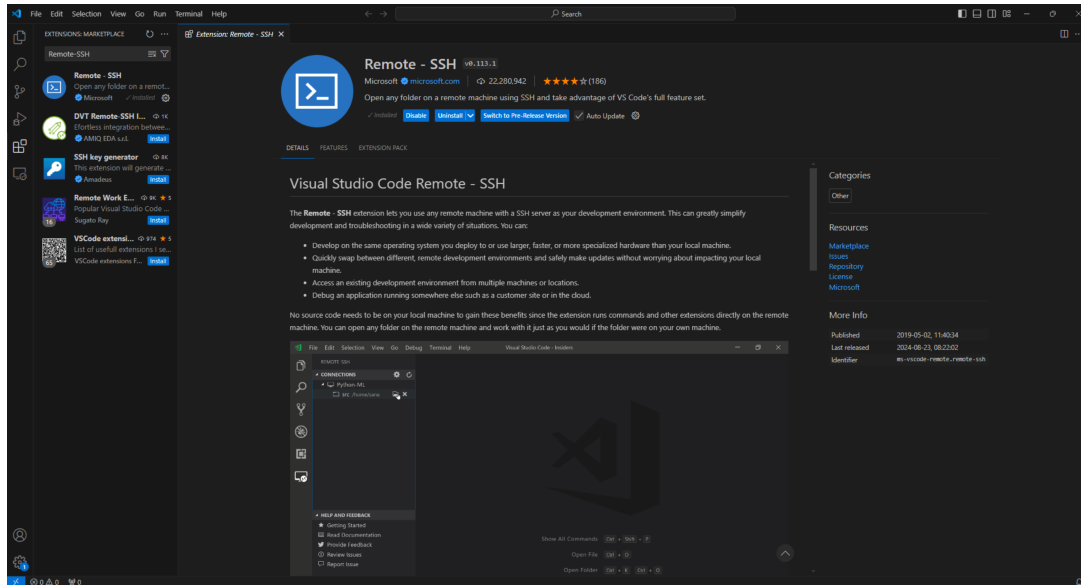


Figure 1: Remote-SSH extension in VS Code

4 Set Up SSH Tunneling on Remote Server

To work on your assignments on the server using VS Code, we need to set up SSH key authentication.

1. Generate a new SSH key pair:

```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```

2. Copy the contents of the SSH public key file (.pub).
3. Log in to the remote server using your UCInetID:

```
ssh your_uci_net_id@bondi.eecs.uci.edu
```

4. Create the .ssh directory if it does not exist:

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
```

5. Add the public key to authorized_keys:

```
echo "<PASTE_PUBLIC_KEY_HERE>" >> ~/.ssh/authorized_keys
```

6. Set correct permissions:

```
chmod 600 ~/.ssh/authorized_keys
```

7. Log out of the server:

```
exit
```

5 Connect to the Remote Server Using VS Code

Step 1: Open SSH Configuration File

- Open the Command Palette:
 - Windows/Linux: `Ctrl+Shift+P`
 - macOS: `Cmd+Shift+P`
- Type `Remote-SSH: Open SSH Configuration File...` and select it
- Edit the file:
 - Linux/macOS: `~/.ssh/config`
 - Windows: `C:\Users\<YourUsername>\.ssh\config`

Step 2: Add a New SSH Host Configuration

- In the SSH config file, add a new entry for your remote server:

```
Host my-remote-server
  HostName bondi.eecs.uci.edu
  User your-username
  IdentityFile ~/.ssh/id_rsa
```

- Replace the following:
 - `my-remote-server` with a friendly name for this connection.
 - `remote.server.com` with the remote server's address (ex. `bondi.eecs.uci.edu`).
 - `your-username` with your UCInetID on the remote server.
 - `~/.ssh/id_rsa` with the path to your private key file (adjust this path if your private key is stored elsewhere).

Step 3: Connect to the Remote Server

1. Open the Command Palette:
 - Press `Ctrl+Shift+P` (Windows/Linux) or `Cmd+Shift+P` (macOS).
2. Connect to Host:
 - Type `Remote-SSH: Connect to Host...` and select it.
 - From the list of hosts, choose the one you just configured (`my-remote-server`).
3. Authenticate:
 - If prompted, enter the passphrase for your SSH key (if you set one during key generation).
 - VS Code will establish an SSH connection to the remote server. This may take a few moments as it sets up the remote environment.
4. Verify Connection:
 - Once connected, you should see `[SSH: my-remote-server]` in the bottom-left corner of VS Code, indicating that you are connected to the remote server.

Step 4: Add tools directory on your path

1. Open Your `.tcshrc` File. Using the code command will open your file on VS Code:

```
code ~/.tcshrc
```

2. Add the following line to the `tcshrc` file to include tools directory.

```
setenv PATH /ecelib/eceware/riscv.old/bin/:$PATH
```

3. Save and close the file
4. To apply the changes immediately, you can source the `tcshrc` file:

```
source ~/.tcshrc
```

6 Running Assembly Code on QEMU

Step 1: Write Assembly Code

Write your RISC-V assembly code using a text editor. Save the code with `.s` extension (e.g., `program.s`).

Step 2: Assemble the Code

Assemble the assembly code using the RISC-V assembler (`riscv64-unknown-elf-as`). Replace `program.s` with your actual assembly file name:

```
riscv64-unknown-elf-as -o program.o program.s
```

Step 3: Link the Object File

Link the object file to create an executable using the RISC-V linker (`riscv64-unknown-elf-ld`):

```
riscv64-unknown-elf-ld -o program.elf program.o
```

Step 4: Create a Binary Image

Create a Binary Image from the elf file to be run in QEMU

```
riscv64-unknown-elf-objcopy program.elf -O binary program.img
```

Step 5: Run the Program on QEMU

Run the assembled program on QEMU using the RISC-V emulator (`qemu-system-riscv64`):

```
qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img
```

Using a MakeFile

A Makefile is a special file used in software development to automate the build process of a project. It contains a set of rules and dependencies that define how the project should be compiled, linked, and assembled. The primary purpose of a Makefile is to specify how different parts of a software project depend on each other and how they should be built or rebuilt when source code changes.

Here are some key concepts associated with Makefiles:

Targets: Targets are the output files or actions that need to be performed. Examples include executable binaries, libraries, or even actions like cleaning up temporary files.

Dependencies: Dependencies are files or actions that the target depends on. If a dependency changes, the associated target needs to be rebuilt.

Rules: Rules define how to create a target from its dependencies. They consist of a target, a colon, a list of dependencies, and the commands to execute for building the target.

Variables: Variables are used to store values that can be reused throughout the Makefile. They can make the Makefile more flexible and easier to maintain.

Makefiles are used with the `make` command, which reads the Makefile and executes the specified targets. They provide a standardized and efficient way to manage complex build processes, ensuring that only the necessary parts of a project are rebuilt when changes occur, saving time and resources during development.

We provided a sample Makefile with this Lab as a reference. The Makefile defines few targets to compile, link and create an image give an assembly file. It also has targets for running and debugging the target. For example, to debug your code, run `make debug`.

For more details on Makefiles, check GNU Make Documentation

7 Examples

In this problem, you will compile/assemble code and debug it on the QEMU processor. For each of the following assembly codes, run each of the Makefile targets by itself to familiarize yourself with the output of each step. For example, try to do the compilation step first, then the linking step, and lastly the image creation step. After you familiarize yourself with each step individually, you can run `make all` to run all the targets.

Part A: Hello World

Below is the assembly code to print Hello World to the UART port of the virtual board:

```
1 1 .global _start
2
3 3 _start:
4
5 6     li s1, 0x10000000      # UART output register
6 7     la s2, hello          # load string start addr into s2
7 8     addi s3, s2, 13        # set up string end addr in s3
8
9 10 loop:
10 11    lb s4, 0(s2)           # load next byte at s2 into s4
11 12    sb s4, 0(s1)           # write byte to UART register
12 13    addi s2, s2, 1         # increase s2
13 14    blt s2, s3, loop       # branch back until end addr (s3) reached
14
15 16 forever:
16 17    j forever
17
18 20 .section .data
19
20 22 hello:
21 23 .string "hello world!\n"
```

To run the code, we can use the Makefile provided and run `make run` in the terminal.